

CloudShield AI / DAIL

11 — Evidence & Logging Specification

Version 1.0 | Final Evidence/Logging Baseline | 1 September 2026

Purpose: Define the immutable evidence model, structured logging taxonomy, lineage and correlation requirements, integrity mechanisms, redaction rules, storage lifecycle, replay/reproducibility requirements, and audit trail required to explain every material DAIL state transition and safety decision.

Document Control

Field	Value
Document ID	CSAI-DAIL-EVID-011
Status	FINAL EVIDENCE/LOGGING BASELINE
Parent documents	01 SRS; 02 Architecture; 03 Functional; 04 Terraform/AWS; 05 Domain/Data; 06 Lifecycle; 07 Identity/Dependency; 08 Impact/Invalidation; 09 Verification/Promotion; 10 LLM Integration
Primary outputs	Evidence Records, Audit Events, Structured Logs
Core principle	No material trust decision without attributable, state-specific evidence.
Integrity model	Content hashes + immutable lineage + append-only event semantics
Sensitive-data principle	Minimize, redact, and never log secrets.

1. Scope

This specification defines the cross-cutting evidence and logging system used by DAIL. It applies to state creation, candidate generation, identity analysis, dependency construction, impact analysis, verification, LLM interactions, promotion, rejection, retries, escalation, and experiment execution.

- Evidence data model.
- Event/log taxonomy.
- Correlation and lineage.
- Integrity and hashing.
- Validity/invalidation.
- Storage and retention.
- Redaction.
- Reproducibility.
- Auditability.
- Failure and recovery logging.

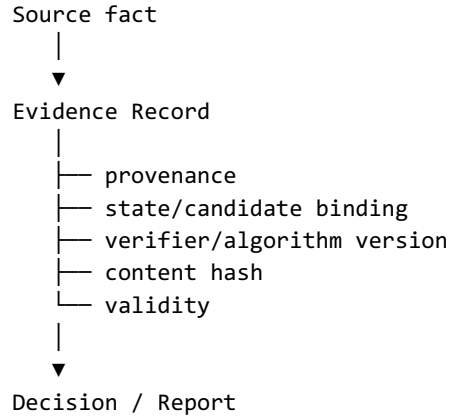
2. Evidence vs Logging

Artifact	Purpose
Evidence	Substantive fact/result used to support a technical decision.
Audit Event	Immutable record that an important lifecycle action

	occurred.
Structured Log	Operational diagnostic record used to observe execution.
Artifact	Large persisted input/output such as Terraform, plan, graph, prompt, or report.
Decision Record	Explicit final interpretation such as PROMOTE/REJECT.

A log saying 'verification passed' is not itself sufficient evidence. The corresponding VerificationResult and supporting evidence must exist.

3. Evidence Trust Model



- Evidence is bound to the state context in which it was produced.
- Evidence has explicit validity rather than implicit permanence.
- Historical evidence is retained even after invalidation.
- A later decision must reference the evidence it relied upon.

4. Evidence Principles

- Every material safety claim is attributable.
- Every evidence record is state-specific where required.
- Evidence provenance is mandatory.
- Evidence content is integrity-protected.
- Invalidation changes validity, not history.
- Evidence cannot be silently overwritten.
- Logs do not replace decision evidence.
- LLM rationale is generation evidence, not safety proof.

5. Evidence Taxonomy

Type	Examples
STATE	Canonical state snapshot, state hash, parent lineage.
CONFIGURATION	Normalized Terraform configuration, plan, resource attributes.
IDENTITY	Resource correspondence/change evidence.
DEPENDENCY	Graph node/edge derivation evidence.
IMPACT	Reason/path showing invariant was affected.

VERIFICATION	Predicate result and supporting observations.
LLM	Prompt metadata, model configuration, response, patch.
PROMOTION	Decision, preconditions, commit/new state.
ERROR	Verifier/parser/provider/system failure evidence.
EXPERIMENT	Trial configuration, baseline, metrics, outcome.

6. Evidence Record Contract

```
EvidenceRecord {
  evidence_id
  evidence_type
  state_id | candidate_id
  parent_state_id
  operation_id
  correlation_id
  source_component
  source_type
  content_hash
  payload_ref
  created_at
  algorithm_or_verifier_version
  validity
  schema_version
}
```

Payloads may be inline for small records or stored as content-addressed artifacts for large objects.

7. Evidence Validity

Status	Meaning
VALID	Eligible for use under current policy/context.
INVALID	Historical evidence retained but no longer valid for the referenced new-state context.
UNCERTAIN	Validity cannot be established safely.
SUPERSEDED	Replaced by a newer authoritative artifact while remaining auditable.
REDACTED	Original content retained only under approved protected storage or replaced by a redacted representation.

8. State Binding

Evidence that can influence trust must identify exactly which state/candidate it describes.

```
state_binding =
  trusted_state_id
  OR candidate_id
  OR explicit operation context
```

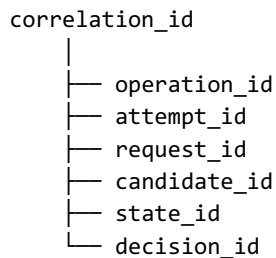
state_hash must be recorded whenever content depends on semantic state.

Evidence without sufficient state binding cannot be used as promotion proof.

9. Candidate Binding

- LLM patches bind to parent Trusted State.
- Impact reports bind to the exact candidate.
- Verification results bind to the exact candidate hash.
- Promotion decisions bind to candidate and parent state.
- A candidate artifact cannot be reused against a different parent without re-analysis.

10. Correlation Model



Correlation IDs allow one complete workflow to be reconstructed across components and asynchronous operations.

11. Identifier Requirements

Identifier	Requirement
correlation_id	One end-to-end workflow lineage.
operation_id	One logical DAIL operation.
attempt_id	One generation/verification attempt where applicable.
request_id	One external/provider request.
candidate_id	One candidate proposal/state.
state_id	One Trusted State or explicit state object.
decision_id	One promotion/rejection decision.
evidence_id	One immutable evidence record.

12. Event Taxonomy

Event	Meaning
STATE_CREATED	State artifact created.
CANDIDATE_CREATED	Candidate State/proposal created.
LLM_REQUESTED	LLM request started.
LLM_COMPLETED	LLM response received.
PATCH_VALIDATED	Generated patch passed deterministic validation.
IDENTITY_COMPLETED	Identity analysis completed.
DEPENDENCY_COMPLETED	Dependency graph completed.
IMPACT_COMPLETED	Impact analysis completed.
VERIFICATION_COMPLETED	Verification completed.
PROMOTION_REQUESTED	Promotion evaluation started.
STATE_PROMOTED	New Trusted State committed.
CANDIDATE_REJECTED	Candidate rejected.
RETRY_SCHEDULED	Retry created.

ESCALATED	Automated decision escalated.
ERROR_OCCURRED	Material error recorded.

13. Audit Event Contract

```
AuditEvent {
  event_id
  event_type
  event_version
  correlation_id
  operation_id
  actor_type
  actor_id | null
  state_id | null
  candidate_id | null
  decision_id | null
  timestamp
  sequence
  payload_hash
  payload_ref
}
```

actor_type may identify system component, user, automation, or provider. Human identity is recorded only when appropriate and permitted.

14. Structured Log Contract

```
LogEvent {
  timestamp
  level
  service
  component
  event_name
  correlation_id
  operation_id
  candidate_id | null
  state_id | null
  duration_ms | null
  status
  error_code | null
  metadata
}
```

- Logs must be machine-readable.
- Free-form messages are supplementary, not the canonical audit record.
- High-cardinality sensitive payloads should be referenced by ID/hash rather than copied into logs.

15. Log Levels

Level	Use
ERROR	Operation failed or integrity/safety boundary was violated.

WARN	Unexpected condition requiring attention but execution may continue.
INFO	Material lifecycle operation.
DEBUG	Diagnostic detail useful during development.
TRACE	Very fine-grained diagnostic data; disabled by default in production.

16. Content Hashing

Every immutable evidence/artifact payload must have a canonical content hash.

```
content_hash = SHA-256(canonical_serialization(payload))
```

- Canonical serialization must be deterministic.
- Hash algorithm/version must be recorded.
- Hash changes whenever semantic payload changes.
- Hash is integrity metadata, not confidentiality.

17. Artifact Identity

Large artifacts such as Terraform snapshots, plans, prompts, responses, graphs, and reports should be stored by content-addressed identity.

```
artifact_ref =
  evidence://<type>/<content_hash>
```

The physical storage backend may differ; the logical reference must remain stable.

18. Hash/Lineage Chain

Audit events may include the previous event hash to make tampering or deletion easier to detect.

```
event_n_hash =
  SHA256(event_n_payload + event_n-1_hash)
```

A chain is an integrity enhancement; the authoritative semantic records must still be independently stored and indexed.

19. Timestamp Model

Timestamp	Purpose
created_at	Artifact creation time.
started_at	Operation start.
completed_at	Operation completion.
observed_at	Time a fact was observed, when distinct.
committed_at	Trusted-state commit time.

Timestamps should be stored in UTC with explicit timezone semantics. Ordering must not rely solely on wall-clock timestamps; sequence numbers are used within an operation/stream.

20. Event Ordering

- Each operation receives monotonically increasing sequence numbers where feasible.
- Concurrent events may share wall-clock timestamps.
- Cross-service ordering uses correlation + sequence/causal metadata.
- Promotion commit order is authoritative for Trusted-State advancement.

21. Evidence Provenance

```
Provenance {  
  source_component  
  source_type  
  source_reference  
  observed_at  
  transformation_steps[]  
  algorithm_version  
}
```

Transformation steps explain how raw configuration became normalized data, graph facts, verification results, or decisions.

22. Evidence for Identity

- Trusted resource reference.
- Candidate resource reference.
- Matching evidence.
- Change classification.
- Normalization version.
- Identity algorithm version.
- Provider/runtime ID evidence where used.
- Decision rationale.

23. Evidence for Dependency

- Source/target semantic IDs.
- Edge type.
- Source fact.
- Target fact.
- Derivation rule.
- Terraform attribute/reference where applicable.
- Dependency extractor version.
- Known/uncertain/unsupported status.

24. Evidence for Impact

- Changed resource.
- Invariant ID/version.
- Impact type.

- Dependency path where applicable.
- Scope match.
- Uncertainty source.
- Prior evidence invalidated.
- Impact algorithm version.

25. Evidence for Verification

```
VerificationEvidence {
  verification_id
  invariant_id
  invariant_version
  candidate_id
  verifier_id
  verifier_version
  predicate
  inputs[]
  observations[]
  result
  evidence_refs[]
  started_at
  completed_at
}
```

The predicate and observations must be sufficient to understand what was evaluated and why the result was produced.

26. Evidence for Promotion

```
PromotionEvidence {
  decision_id
  candidate_id
  parent_state_id
  required_checks[]
  verification_refs[]
  evidence_sufficiency
  policy_version
  decision
  reason
  new_state_id | null
}
```

PROMOTE must reference every mandatory verification result used to establish eligibility.

27. LLM Evidence

Artifact	Required metadata
Prompt	Prompt version, context hash/version, request ID.
Model request	Provider, model/deployment, parameters, request ID.
Response	Response hash, parse status, request linkage.
Patch	Patch hash, parent state, candidate linkage.

Rationale	Stored as generation metadata; not safety proof.
Token usage	Record when provider exposes it.

28. Redaction Rules

- Never log plaintext credentials, API keys, private keys, access tokens, passwords, or session secrets.
- Do not log full secret-bearing Terraform values.
- Redact secret-like provider attributes before persistence.
- Prefer hashes/references for sensitive large payloads.
- Preserve enough non-sensitive context to reproduce the decision.
- Redaction policy version must be recorded where transformation occurs.

29. Secret Detection

The logging/evidence pipeline should apply deterministic secret-detection rules before persistence.

- Provider credentials.
- Private key material.
- Authorization headers/tokens.
- Known secret attribute names.
- Environment-derived credentials where exposed.
- Future secret classes added by policy.

Detection is a safety net, not permission to place secrets into prompts or logs.

30. Sensitive Artifact Storage

- Raw prompts/responses may require restricted storage.
- Redacted operational logs may be broadly accessible than raw artifacts.
- Access control should follow least privilege.
- Artifact references should not expose secret-bearing content.
- Deletion/retention must follow approved policy without breaking required audit lineage.

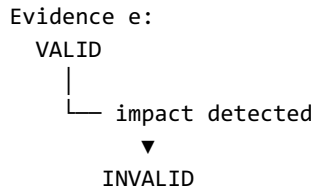
31. Evidence Retention

Category	Default policy
Promotion/rejection decisions	Retain for the full experiment/project audit period.
Trusted State snapshots	Retain for lineage and reproducibility.
Verification evidence	Retain with the associated state/candidate lineage.
LLM raw responses	Retain according to experiment/privacy policy; hashes/metadata retained longer where appropriate.
Operational debug logs	Shorter retention permitted.
Errors	Retain long enough for incident and experiment analysis.

Exact retention duration is deployment policy; semantic audit dependencies must not be broken by premature deletion.

32. Evidence Invalidation

When impact analysis invalidates evidence, the original record remains immutable and a validity-transition event is appended.



Original content remains preserved.

- Record invalidation reason.
- Record affected candidate/state.
- Record impact report reference.
- Do not rewrite the original evidence payload.

33. Evidence Supersession

A newer valid evidence record may supersede an older one without deleting the older record.

evidence_v1 → SUPERSEDED_BY → evidence_v2

This is useful when a verification is repeated or a corrected evidence artifact is generated.

34. Replay/Reproducibility

A replay should be able to reconstruct the logical workflow from persisted state, configuration, versions, hashes, and artifacts.

- Trusted-state snapshot/hash.
- Candidate patch/hash.
- Impact report.
- Dependency graph/version.
- Invariant registry version.
- Verifier version/configuration.
- LLM request metadata where applicable.
- Promotion policy version.
- Evidence references.
- Event sequence.

35. Replay Boundary

Replay is not necessarily a second execution against live AWS. The system must distinguish historical reconstruction from fresh verification.

Mode	Meaning
------	---------

RECONSTRUCT	Rebuild historical reports from stored artifacts.
REVERIFY	Run current verifier against historical/captured state.
REGENERATE	Run LLM generation again under a new or identical experimental condition.

These modes must not be conflated in audit reports.

36. Error Evidence

```
ErrorEvidence {
  error_id
  operation_id
  component
  error_class
  error_code
  sanitized_message
  stack_reference | null
  input_refs[]
  recoverability
  retry_count
  created_at
}
```

- Sanitize exception messages before persistence.
- Preserve enough diagnostic detail for debugging.
- Classify recoverability deterministically where possible.
- Link the error to the failed operation.

37. Provider/API Logging

- Record provider request IDs when available.
- Record status/latency/token metadata where appropriate.
- Do not log authorization headers.
- Map provider errors to normalized DAIL error classes.
- Preserve retry/attempt linkage.

38. Idempotency and Duplicate Events

Repeated delivery of the same event must not create contradictory semantic history.

- Use event IDs/idempotency keys.
- Consumers should deduplicate by immutable event identity.
- A duplicate event may be logged operationally but must not create a second state transition.
- Promotion commit remains transactional.

39. Failure Recovery

1. Persist material failure evidence before retry when possible.
2. Link retry to original operation/attempt.
3. Do not erase failed attempts.
4. Re-run only within configured retry policy.

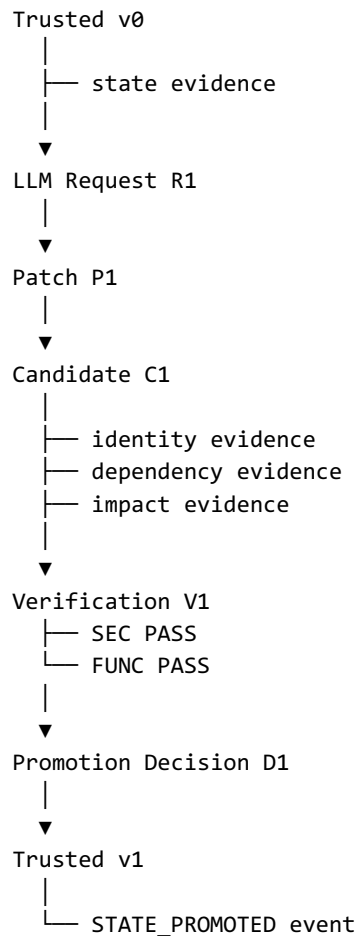
5. Record final outcome separately from intermediate failures.
6. Ensure previous Trusted State remains recoverable.

40. Audit Query Requirements

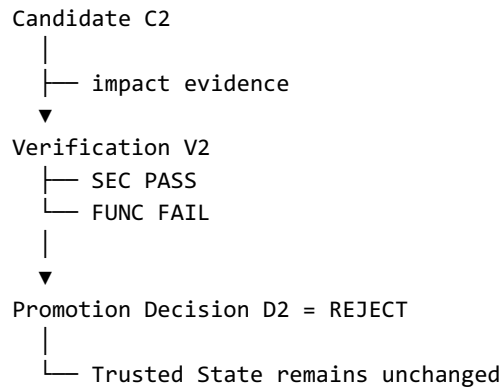
The system should support queries such as:

- Why was Candidate C rejected?
- Which invariant caused rejection?
- Which resource change affected that invariant?
- Which dependency path caused invalidation?
- Which verifier produced the PASS/FAIL?
- What evidence was used for promotion?
- Which LLM request generated the patch?
- Which prompt/model configuration produced it?
- What was the Trusted State parent?
- What exact artifact hashes were committed?

41. Canonical Evidence Chain — Safe Promotion



42. Canonical Evidence Chain — Rejection



43. Evidence Completeness Rule

A material decision is complete only when the required evidence references exist and are internally consistent.

```
decision_complete =  
  decision exists  
  AND candidate binding valid  
  AND parent binding valid  
  AND required verification refs exist  
  AND evidence hashes validate  
  AND policy version recorded
```

44. Consistency Checks

- Evidence state hash matches referenced state.
- Candidate hash matches candidate artifact.
- Impact graph version matches evidence.
- Verification candidate hash matches candidate.
- Promotion decision parent matches current state at commit.
- Evidence references resolve.
- Hashes recompute successfully.
- Event lineage is internally consistent.

45. Tamper Detection

On read or audit, content hashes should be recomputed and compared with stored hashes.

```
verify_integrity(artifact):  
  canonicalize(artifact)  
  recompute SHA-256  
  compare stored hash  
  return VALID | TAMPERED
```

A tampered evidence artifact must never silently remain eligible for promotion proof.

46. Integrity Failure Handling

- Mark artifact/evidence integrity failure.
- Emit ERROR_OCCURRED audit event.
- Invalidate affected evidence for decision purposes.
- Block promotion if required evidence is affected.
- Escalate for investigation when appropriate.
- Preserve the integrity-failure record.

47. Access and Authorization Boundary

- Read access to evidence is controlled by environment/security policy.
- Write access to authoritative evidence is restricted to owning components.
- Promotion records cannot be edited by the LLM adapter.
- Historical evidence is append-only from the application perspective.
- Administrative corrections must themselves be auditable events.

48. Experiment Logging

Because DAIL is intended to evaluate AI-assisted infrastructure repair, experiment records must preserve enough information to compare methods fairly.

- Experiment ID.
- Trial ID.
- Baseline state.
- Candidate generation condition.
- Model configuration.
- Prompt/context versions.
- Patch hash.
- Verifier versions.
- Impact/verification outcomes.
- Latency/token usage where available.
- Final decision.
- Failure/retry counts.

49. Metrics

Metric	Purpose
Generation latency	LLM efficiency.
Generation attempts	Retry/robustness analysis.
Patch acceptance rate	Candidate-generation quality.
Verification PASS/FAIL rate	Repair quality.
Promotion rate	End-to-end candidate success.
Rejection reason distribution	Failure analysis.
Unknown/unsupported rate	Coverage limitations.
Evidence invalidation count	Impact behavior.
Token usage	Generation cost/efficiency.
End-to-end cycle time	System efficiency.

50. Logging Anti-Patterns

- Logging only 'PASS' without predicate/evidence.
- Logging raw secrets for debugging.
- Overwriting previous decisions.
- Using timestamps as the only lineage mechanism.
- Dropping failed attempts.
- Treating logs as authoritative state.
- Persisting LLM confidence as proof.
- Storing artifacts without hashes.
- Reusing evidence without state/version compatibility.

51. Testing Strategy

51.1 Evidence Tests

- Schema validation.
- Hash stability.
- State binding.
- Candidate binding.
- Invalidation transition.
- Supersession.
- Tamper detection.
- Missing-reference detection.

51.2 Logging Tests

- Event schema.
- Correlation propagation.
- Sequence ordering.
- Duplicate event handling.
- Error classification.
- Secret redaction.

51.3 End-to-End Tests

- Safe promotion evidence chain.
- Rejected candidate evidence chain.
- Retry chain.
- Escalation chain.
- Stale parent chain.
- Verifier error chain.
- LLM generation-to-promotion lineage.

52. Acceptance Criteria

- Every material DAIL decision has attributable evidence.
- Evidence records are immutable and hash-protected.
- Evidence is bound to exact state/candidate context.
- Impact invalidation preserves history and changes validity explicitly.

- Promotion references sufficient verification evidence.
- Audit events can reconstruct end-to-end lineage.
- Logs are structured and correlated.
- Secrets are redacted before persistence.
- Duplicate events cannot create duplicate semantic transitions.
- Integrity failures block use of compromised evidence.
- Experiment records contain enough metadata for fair comparison and replay.
- Failed/retried/rejected attempts remain auditable.

53. Non-Goals

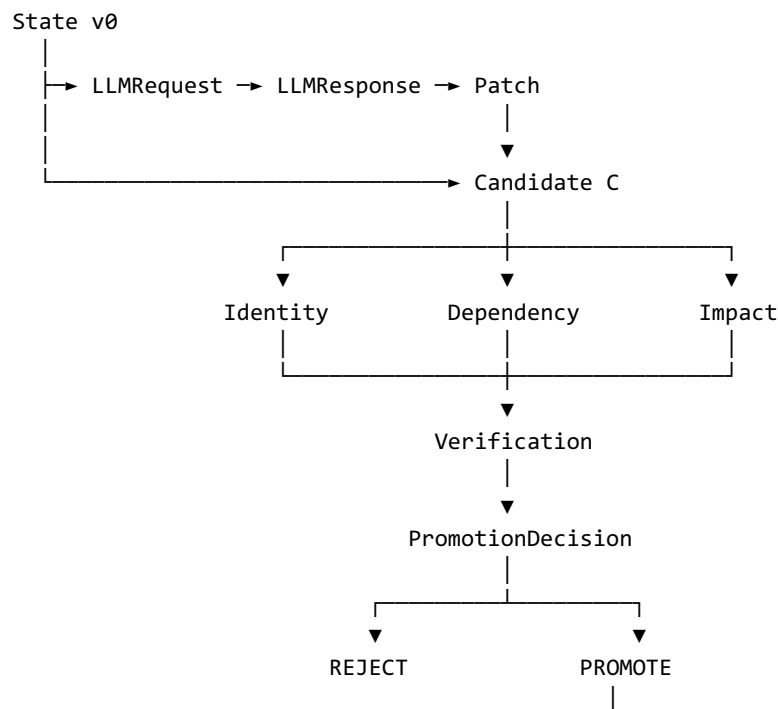
- Replacing a database transaction system.
- Defining cloud-provider security controls.
- Guaranteeing absolute tamper resistance without infrastructure controls.
- Storing every runtime detail forever.
- Using logs as substitutes for verification evidence.

54. Final Evidence/Logging Directive

DAIL shall treat evidence as a first-class engineering artifact. Every material state transition, analysis result, verification result, and promotion decision must be attributable to a specific state/candidate context, component, version, and integrity-protected evidence chain.

Logging provides operational visibility; evidence provides decision support; audit events provide lifecycle history. These layers must remain distinguishable while sharing correlation and lineage so that the complete path from LLM proposal to Trusted State can be reconstructed.

Appendix A — Minimal End-to-End Evidence Graph



Appendix B — Next Specification

Document 12 — Testing & QA Specification defines the complete validation strategy for DAIL, including unit/integration/system tests, adversarial fixtures, regression tests, verifier/oracle comparison, experiment correctness, CI gates, coverage expectations, and release acceptance.